

Piper 抓取流程安全设置总结

基于当前 apps/piper_run_test.py、core/robot_safety.py 及其直接依赖代码的现状审计

生成日期：2026-09-09 项目：VLM-6DoF-Grasp

文档范围

本文只总结当前代码已经实现的检查、反馈确认、失败恢复和已知边界，不修改任何项目代码，也不把改进建议描述为现有能力。

1. 总体安全链路

流程采用“先确认状态、再启用恢复、分阶段感知和运动、每条命令反馈验收、任何异常回 HOME”的结构。主要调用顺序如下：

- 加载并验证硬件配置与应用组件配置。
- 建立 Piper LCM 客户端，但此时不自动发送 HOME。
- 等待 ARM_STATE，检查急停，再启用 safe-stop 生命周期。
- 发送 HOME，并通过状态反馈确认到位。
- 初始化相机、FFS、检测、分割、EconomicGrasp、selector 和 executor；执行相机首帧握手。
- 远距离观察与目标定位 → 近距离观察 → 再取新 RGB-D → 检测、分割、抓取生成与候选选择。
- 将相机系抓取位姿转换到基座系，依次执行 approach、reach、grasp、lift、home。
- 任一步异常进入 finally；若安全恢复仍启用，则尝试回 HOME，随后逆序释放资源。

2. 初始化与任务准入

2.1 配置完整性

- validate_hardware() 强制要求 home_pose、ready_views、target_approach_offset 和 target_approach_rpy 均已配置；缺失时任务在机械臂运动前终止。
- GraspManager 检查应用声明的 robot backend 是否与硬件 profile 的 robot.kind 一致，防止错误驱动组合。
- grasp executor 配置 require_home: true；若抓取步骤为空或未步不是 HOME，执行器构建失败。

2.2 状态优先于运动

- piper_lcm 组件创建 SafeRobotClient 时 safe_stop_enabled=False，避免尚未确认 ARM_STATE 就自动回 HOME。
- wait_for_robot_state() 在 state-timeout 内等待首个 ARM_STATE；默认 10 秒，超时禁止继续。
- require_not_emergency_stopped() 检查 arm_status==1；急停状态要求人工解除后重新运行。
- 通过上述检查后才调用 enable_safe_stop()，随后 reset_to_home_and_wait() 建立已知 HOME 基线。

3. 取帧与感知阶段的检查

3.1 相机与 RGB-D

- 感知组件加载失败会被包装为“感知初始化失败”，不会进入运动任务。
- manager.handshake() 反复调用 camera.step()，直到取得首张彩色帧或超时。
- capture_rgbd() 每次先清空 ctx.color、ctx.depth、ctx.ir，避免把旧帧当作本次结果。
- camera.capture() 默认丢弃配置中的 5 帧陈旧数据；随后 depth.step() 生成本次深度。
- RGB 或 depth 任一为空时，统一 retry() 最多尝试 3 次，仍失败则终止。

3.2 远距离目标定位

- 先到固定远观察位姿，再检测目标，避免直接根据未知近景移动。
- 检测必须返回至少一个 box；否则目标定位重试，最多 3 次。
- box_center_to_base() 从框中心 5×5 邻域取正深度的中位数，降低单像素坏点影响。
- 中心邻域没有有效深度时返回 None，由 retry() 阻止继续接近。
- 有效像素经相机内参、手眼外参和当前末端位姿转换为基座系目标点。

3.3 分割与抓取候选

- 检测框围绕中心扩大 1.5 倍并裁剪至图像范围，再作为 EfficientSAM box prompt。
- 分割为空时最多重试 3 次；RGB、depth、mask 尺寸必须完全一致。
- EconomicGrasp 必须返回非空候选；配置 use_collision:true，启用点云模型碰撞过滤。
- 候选可按姿态角过滤，并限制进入 selector 的数量；当前配置 predict_topk=100、selector_topk=12。
- 检测图、RGB-D、mask 和候选图通过 try_save() 保存；保存失败只记录，不中断核心安全流程。

4. EconomicGrasp 原始候选与 first 筛选细节

当前抓取候选不是由 first selector 直接从网络原始输出中任取。完整顺序为：目标 mask 点云 → EconomicGrasp 网络预测 → 碰撞过滤 → NMS → 置信度排序 → top-k → 3D 姿态筛选 → first 二维几何筛选 → PCA 选择。

4.1 EconomicGrasp 输入点云筛选

- 由 depth 和相机内参建立有组织点云；workspace_mask 使用 EfficientSAM mask，随后再与 depth>0 相交。mask 外点和无效/零深度点不会进入网络。
- mask 与 depth 尺寸不一致时直接报错；目标 mask 内没有有效点时返回空 GraspGroup，外层重试后仍为空则终止。
- 有效点不少于网络 num_points 时无放回随机采样；不足时保留全部点并有放回补齐。因此点少并不等于拒绝，但重复采样可能降低几何可靠性。
- 网络 pred_decode() 输出每个 6DoF 抓取的 score、width、height、depth、rotation 和 translation，形成原始 GraspGroup。

4.2 EconomicGrasp 内部后处理

- 碰撞筛选：use_collision=true 且 collision_thresh>0 时，ModelFreeCollisionDetector 基于采样目标点云检测夹爪碰撞；approach_dist 固定为 0.05 m，collision_mask 为真的候选被删除。
- NMS：gg.nms() 抑制空间位置和姿态相近的重复抓取，避免大量近乎相同候选占据 top-k。NMS 的具体距离/角度阈值由 GraspNetAPI 实现提供，本应用没有额外覆盖。
- 置信度排序：gg.sort_by_score() 将剩余候选按 EconomicGrasp score 从高到低排列。
- 数量截断：当前 pipeline 调用 topk=100，所以最多把前 100 个高分候选返回 GraspPerception。此时所谓“第一个原始候选”是碰撞过滤、NMS 和排序后的最高分候选，不是网络张量中的第一个元素。

碰撞过滤边界

这里检查的是夹爪几何与当前采样点云的局部碰撞，不检查机械臂连杆、运动轨迹、相机盲区或点云缺失区域，因此不能代替整臂路径规划。

4.3 第一阶段 3D 姿态筛选

GraspPerception.generate() 在 EconomicGrasp 返回后调用 filter_grasps_by_orientation()。它按已经排好的 score 顺序扫描，最多保留 selector_topk=12 个。

- angle_x = arccos(rotation[:,0] · [0,0,1])：比较抓取局部 X 轴（接近方向）与相机 Z 轴，要求 angle_x<50°。

- $\text{angle_y} = \arccos(\text{rotation}[:,1] \cdot [1,0,0])$: 比较抓取局部 Y 轴（夹爪开合方向）与相机 X 轴，要求 $\text{angle_y} < 100^\circ$ 。
- 只有两个角度条件同时成立才进入 keep; 扫描到 12 个合格候选后停止。候选仍保持原 score 顺序。
- 如果一个候选都不合格，函数不会返回空，而是回退到原排序的前 12 个候选。故这是方向偏好筛选，不是强制拒绝。

4.4 first selector 的二维投影

FirstGraspSelector 调用 `vlm_grasp_visualize_batch()`，把最多 12 个 3D 候选投影到 RGB 图像。每个候选投影 5 点：两侧指尖 `pts[0]/pts[3]`、两侧指根 `pts[1]/pts[2]`、EconomicGrasp translation 中心 `pts[4]`。
。 $t[2] \leq 0$ 的相机后方候选直接跳过。

- TooNarrow: $\text{width_px} = ||\text{pts}[0] - \text{pts}[3]|| < 55$ 像素时拒绝，防止图像中夹爪投影开口过窄。该阈值是像素阈值，会随深度、分辨率和内参变化。
- BadPose: $\text{pts}[2].x < \text{pts}[1].x$ 且 $\text{pts}[2].y > \text{pts}[1].y$ 时拒绝。这是针对当前相机/夹爪投影方向的经验象限规则。
- BadAngle: 计算指根连线 $\text{vec_base} = \text{pts}[2] - \text{pts}[1]$ 与指根中心到指尖中心箭头 vec_arrow 的夹角，并折算为 $0 \sim 90^\circ$ ； $\text{angle_deg} < 45^\circ$ 时拒绝。
- PointingDown: $\text{center_tip}.y > \text{center_base}.y$ 时拒绝。图像 Y 向下增加，因此它拒绝投影箭头朝图像下方的候选。
- 上述四个条件使用 OR: 任意一个为真就拒绝并保存 REJECT 调试图；全部为假才重新编号为有效 candidate。

4.5 first selector 的 PCA 最终选择

- 从目标 mask 提取所有 $[x, y]$ 像素并计算 2×2 协方差矩阵；少于 3 个像素时直接选择有效候选 0。
- `np.linalg.eigh()` 的特征值从小到大排列。 $\text{ratio} = \sqrt{\text{大特征值} / \text{小特征值}}$ ，近似目标长轴/短轴比例。
- $\text{ratio} \leq 1.3$ 时认为接近圆形/正方形，使用固定水平轴 $[1, 0]$ ； $\text{ratio} > 1.3$ 时认为细长，使用 `vectors[:, 0]`，即小特征值对应的目标短轴。
- 对每个有效候选取 $\text{base} = \text{pts}[2] - \text{pts}[1]$ 作为二维夹爪开合轴， $\text{score} = |\text{base} \cdot \text{axis}| / ||\text{base}||$ 。绝对值忽略轴的正反方向。
- `np.argmax(scores)` 选择与目标短轴（或圆形目标水平轴）最平行的候选。返回的是有效 candidates 中的索引，不等于 EconomicGrasp 原始 `source_index`。

4.6 first 的全部拒绝兜底

- 二维循环会备份第一个满足 $t[2] > 0$ 的候选及其 `source_index`。

- 若四项二维规则把所有候选都拒绝，则强制恢复该备份作为 ID 0，并附加 `fallback_rpy=[0, 0.85, 0]`。
- `execute_selected()` 后续会用 `fallback_rpy` 覆盖位姿转换得到的 RPY，因此兜底不仅恢复候选，也改变最终末端姿态。
- `selector` 最后仍检查 `candidates` 非空和 `index` 范围；完全没有可投影候选时会终止，不发送抓取动作。

安全结论

EconomicGrasp 碰撞过滤和多个方向/二维条件能减少明显不合适候选，但 3D 姿态筛选与二维筛选都存在“全部失败后保留候选”的设计。`first` 应视为启发式选优，而不是独立的硬安全门禁。

5. EfficientSAM 专项检查

EfficientSAM 组件自身只保证输入/输出的基本有效性；更严格的 mask 质量检查位于 `piper_run_test.py` 的 `sam_test()` 专项模式。

- 组件层：`box` 为空返回 `None`；按预测 IoU 排序取每个 `box` 的最佳 mask；多 `box` 合并；mask 尺寸必须等于 RGB 网格。
- SAM 模式 `containment = mask` 中落入 `box` 的像素 / mask 总像素，要求至少 0.70，用于限制 mask 向框外扩散。
- SAM 模式 `coverage = mask` 与 `box` 交集像素 / `box` 面积，要求至少 0.10，用于拒绝极小碎片。
- SAM 模式最多持续测试 30 秒，并保存检测框与分割结果。

正常 grasp 模式的边界

`containment/coverage` 当前只在 `--mode sam` 中执行；正常 `--mode grasp` 只检查 mask 非空、三者尺寸对齐以及能否生成非空抓取候选。

6. core/robot_safety.py：常量与错误识别

6.1 到位与保持阈值

- `GRIPPER_TOLERANCE_M = 0.002`：夹爪反馈验证容差 2 mm。命令始终发送；该常量决定是否启用到位验证及验证误差阈值。
- `POSITION_TOLERANCE_M = 0.015`：末端位置到位容差 15 mm。
- `ORIENTATION_TOLERANCE_RAD = 5.5°`：末端姿态到位容差。
- `STABLE_TIME_S = 0.3`：满足位姿/夹爪条件后必须连续稳定 0.3 秒。

- `HOLD_POSITION_M = 0.03`、`HOLD_ORIENTATION_RAD = 10°`：保持测试允许的漂移阈值。

6.2 ARM 状态错误

`_read()` 每次读取 `ARM_STATE`；通信中断或 `arm_status` 非零立即抛错。已映射的状态包括：

- 1 急停；2 无逆解；3 奇异点；4 目标位置超限；5 关节通信异常；6 关节抱闸未释放；7 发生碰撞；8 示教超速；9 关节状态异常；10 控制器异常；14 主控过温；15 电阻过温。

7. robot_safety.py 函数与 piper_run_test.py 调用对应

7.1 wait_for_robot_state(robot, timeout)

- 直接调用：`initialize_system()`，创建 `robot` 后立即调用。
- 实现：轮询 `get_state()`，在超时前收到任意有效状态才放行；默认 `state-timeout=10` 秒。
- 效果：没有反馈通信时禁止启用自动恢复、禁止 HOME、禁止加载后续抓取流程。

7.2 require_not_emergency_stopped(state)

- 直接调用：紧接 `wait_for_robot_state()`。
- 实现：`arm_status==1` 时抛错并要求人工解除。
- 效果：不会在已知急停状态下自动发送 HOME 或抓取动作。

7.3 reset_to_home_and_wait(robot, timeout, home_pose)

- 直接调用：初始化安全基线；reach 测试结束；SAM 测试通过后。
- 实现：通过 `_command_and_wait()` 记录旧 `target_utiltime`、发送 `reset_to_home`，再调用 `_wait_for_arrival()` 验证 HOME。
- 效果：程序不是“发出 HOME 就当成功”，而是确认控制器接收新目标并实际稳定到位。

7.4 move_to_pose_and_wait(robot, hw, pose, gripper, timeout)

- 直接调用：远距离观察、近距离观察、SAM 模式的两次观察移动。
- 间接调用：`execute_selected()` → `GraspExecutor.run_sequence()` 的每个 `approach/reach/grasp/lift/home` 步骤。
- 实现：发送前调用 `hw.in_workspace()` 检查目标；再由 `_command_and_wait()` 和 `_wait_for_arrival()` 做反馈验收。
- 效果：所有主要笛卡尔运动共享同一套工作空间、通信、控制器状态、到位误差与超时检查。

8. robot_safety.py 的反馈验收细节

8.1 _command_and_wait()

- 普通命令发送前最多连续轮询 ARM_STATE 3 秒（每次间隔 10 ms）；持续没有状态才拒绝发送。安全恢复的 send_without_state=True 不等待这 3 秒。
- 记录发送前 target_utime，用来判断控制器是否接收了一个新目标。
- 仅当夹爪当前反馈存在且目标变化达到 2 mm 时，才设置 expected_gripper；小于 2 mm 仍发送命令，只是不额外等待夹爪反馈。
- 命令发送后统一进入 _wait_for_arrival()。

8.2 _wait_for_arrival()

- 循环内每次调用 _read()，持续检查通信中断和 arm_status 错误。
- target_utime 必须不为 0 且不同于发送前值，否则视为控制器尚未接受命令。
- 首次接受的新 target_utime 被锁定；等待过程中出现另一个目标时间戳，判定命令被覆盖并立即失败。
- 位置误差按三维欧氏距离计算；姿态误差通过两个旋转的相对旋转幅度计算。
- 位姿与需要验证的夹爪反馈同时合格，并连续稳定 0.3 秒才返回成功。
- 超时错误会区分：控制器未接收、位姿未到、夹爪未到、或未连续稳定，并报告实际误差/当前位姿。

8.3 工作空间检查

- move_to_pose_and_wait() 在发送前调用 hw.in_workspace(x, y, z)。当前 piper_d405.yaml 配置 X=[0, 0.75] m、Y=[-0.65, 0.65] m。

现有限制

硬件 profile 没有配置 z_max，因此当前实际只检查目标点 X/Y；也不检查从当前位置到目标点的整条扫掠路径。

9. 其余 robot_safety.py 调用

9.1 monitor_robot_state()

- 直接调用：仅 --mode reach，在到达 reach 位姿后保持 30 秒。
- 实现：持续调用 _read()，并比较实际位姿与期望位姿；位置漂移超过 30 mm 或姿态漂移超过 10° 立即失败。
- 效果：测试期间即使 motion_status 不可靠，也通过实际末端位姿判断是否稳定。

9.2 safe_stop_and_wait()

- 直接调用: `initialize_system()` 的 `finally`, 只要 `safe-stop` 已启用, 无论成功、感知失败还是运动失败都会进入。
- 实现: 若当前 `arm_status==1`, 则关闭 `safe-stop` 标记且不再发送 HOME; 否则调用 `robot.safe_stop()` 并等待 HOME 到位。
- `send_without_state=True` 允许在最终恢复阶段先发送 `safe_stop`, 但后续仍需 `ARM_STATE` 才能完成到位验收。
- 效果: 异常时尽量回到已知 HOME; 急停优先级高于自动恢复, 避免程序覆盖人工急停。

9.3 内部辅助函数

- `_values()`: 兼容 `ee_pose` 与 `tcp_pose` 字段。
- `_pose()`: 格式化当前位姿, 供日志和异常报告。
- `_pose_error()`: 计算位置距离和旋转角误差。
- `_read()`: 所有等待与保持监控的统一状态门。

10. 抓取执行阶段

10.1 位姿转换与输入检查

- `execute_selected()` 在转换前再次获取 `ARM_STATE`; 无状态禁止执行。
- `convert_new()` 组合抓取位姿、轴对齐、`grasp_depth` 指尖补偿、手眼标定和当前末端位姿。
- `grasp_depth` 必须是有限且非负数, 否则转换失败。
- `adjust_ry()` 对 $0\sim 0.9$ rad 范围内的 `Ry` 进行项目特定修正; 候选提供 `fallback_rpy` 时会覆盖转换姿态。
- 闭合宽度为 `selected width - 0.04 m`, 并裁剪到 `[0, gripper_max_width]`。

10.2 配置化五步序列

- `approach`: 相对抓取点偏移 $[-0.06, 0, +0.05]$ m, 夹爪张开。
- `reach`: 到达目标抓取位姿, 夹爪张开。
- `grasp`: 保持抓取位姿并闭合到目标宽度, 等待 1 秒。
- `lift`: 在目标基础上抬升 0.08 m, 保持目标夹爪宽度。
- `home`: 使用硬件 HOME 位姿结束序列。

每一步由 GraspExecutor 捕获异常；任一步失败即返回 (False, reason)，execute_selected() 再将其提升为任务异常，从而触发 finally 安全恢复。

11. 异常恢复与资源生命周期

- initialize_system() 用 contextmanager 包住整个任务；所有 BaseException 都会记录并继续进入 finally。
- 只在 ARM_STATE 与急停检查完成后把 safe=True，因此未建立安全基线时不会盲目执行恢复运动。
- 若 safe 且 SafeRobotClient.safe_stop_enabled 仍为真，finally 调用 safe_stop_and_wait()。
- 正常 grasp 的配置末步已经 HOME；成功后 disable_safe_stop()，避免资源释放阶段重复发送 HOME。
- reach 和 SAM 模式在显式 reset_to_home_and_wait() 成功后同样关闭 safe-stop。
- GraspManager.release_resources() 先对已构建组件调用 safe_stop，再按构建逆序调用 close；单次关闭具备幂等保护。
- Dashboard 和调试文件保存故障被隔离，不会阻断机械臂安全控制和资源清理。

12. 三种运行模式的安全行为

12.1 --mode grasp

- 执行完整远观测、近观测、感知、筛选以及 approach→reach→grasp→lift→home。
- 仅在完整序列成功后关闭自动安全恢复。

12.2 --mode reach

- 复用完整感知和候选选择，但只执行 executor 的前两步 approach、reach。
- 保持 30 秒并监控漂移；随后显式回 HOME，再关闭自动安全恢复。

12.3 --mode sam

- 不加载 grasp_engine、selector 和 executor，不执行抓取。
- 仍会远观测定位、近观测，然后在 30 秒内验证 containment/coverage；通过后回 HOME。

13. 已知安全边界（不是已实现保护）

- 工作空间当前只有 X/Y 边界，没有 Z 下限或上限。
- 只验证离散目标位姿，不做整臂轨迹规划、扫掠体检查或环境路径避障。
- Piper adapter 只发送目标位姿和夹爪宽度，没有在本项目层显式设置速度、加速度、力或夹持力上限。

- GraspExecutor 只对 gripper:max 的步骤启用夹爪到位验证；grasp 目标闭合宽度不验证实际反馈。
- 候选方向和二维几何过滤都有“全部失败后保留候选”的兜底，不能当作硬安全门。
- EconomicGrasp 的碰撞过滤基于目标点云/夹爪模型，不等价于机械臂全身路径碰撞检查。
- 正常 grasp 模式未强制执行 SAM containment/coverage 阈值。
- 自动回 HOME 假设 HOME 指令及其运动路径安全；项目层没有为回程规划绕障路径。
- adjust_ry() 和 fallback_rpy 是项目经验规则，不是机械可达性或无碰撞的形式化证明。

14. 运行前人工核对清单

- 确认急停按钮可用，操作者能够随时触达。
- 确认工作区无人、无临时障碍物，HOME 与观察位姿路径畅通。
- 确认手眼标定、相机内参、单位和 Euler 角约定仍适用于当前硬件安装。
- 确认 piper_d405.yaml 的 HOME、ready view、工作空间和夹爪最大宽度与实机一致。
- 首次测试优先使用 --mode sam，再使用 --mode reach，最后才使用 --mode grasp。
- 降低速度并进行空载/软物体测试（速度设置位于外部 Piper 控制器时，也应在控制器侧确认）。
- 观察每一步 [发送]、[到达] 和 ARM_STATE；发生异常时先使用硬件急停，再排查软件日志。

15. 代码索引

- apps/piper_run_test.py: 任务编排、模式分支、感知与安全函数调用入口。
- core/robot_safety.py: 状态读取、工作空间门禁、命令接受/覆盖检查、反馈到位、保持监控和 HOME 恢复。
- core/grasp_perception.py: 新帧获取、统一三次重试、检测/分割/抓取生成和尺寸一致性检查。
- core/grasp_selector.py: 二维候选几何淘汰、PCA 方向选择及兜底行为。
- core/grasp_executor.py: 配置化分步执行和逐步反馈等待。
- core/transform.py: 相机系抓取姿态到基座系的变换与 grasp_depth 合法性检查。
- core/manager.py: 依赖构建、相机握手、安全停止和逆序资源释放。
- config/apps/piper_run_test.yaml: 组件组合、抓取步骤、碰撞过滤和候选数量。
- config/hardware/piper_d405.yaml: 手眼标定、HOME/观察位姿、夹爪宽度和工作空间边界。

结论

当前实现的强项是状态先行、每步反馈验收、命令覆盖检测和统一异常回 HOME；主要边界集中在路径级碰撞、Z 轴工作空间、闭合夹爪反馈以及候选兜底策略。